T.C YEDITEPE UNIVERSITY FACULTY OF COMMERCE DEPARTMENT OF MANAGEMENT INFORMATION SYSTEMS

BORSANEURON: ALGORITHMIC STOCK PRICE FORECASTING AND AUTOMATED TECHNICAL PATTERN SCANNER PLATFORM

https://github.com/zachariatrying/borsaneuron

Graduation Thesis

by

■brahim Tatar

20211314007

Thesis Supervisor: Dr. Ö■r. Üyesi U■UR TEVF■K KAPLANCALI

ISTANBUL, SPRING 2026

CONTENTS

ÖZ

Bu çal■■ma, Borsa ■stanbul (BIST) hisse senedi piyasalar■nda i■lem gören hisse senetlerinin teknik analiz indikatörleri ve geometrik fiyat örüntülerini kullanarak, 5 günlük gelecek fiyat yönünü tahmin etmeyi ve klasik grafik formasyonlar■n■ otomatik olarak taramay■ hedefleyen bütünsel bir karar destek platformu olan **BorsaNeuron**'u sunmaktad■r.

Geli■tirilen bu proje, Borsa ■stanbul genelini temsil eden **aktif BIST hisse senetlerine** ait tarihsel günlük veriler üzerinde yürütülmü■tür. Modelleme verimlili■ini ve do■rulu■unu art■rmak amac■yla, 30 teknik indikatör de■i■keninden olu■an zengin bir özellik seti olu■turulmu■ ve gelecek s■z■nt■s■ (look-ahead bias) engellenmi■tir. De■i■kenler aras■ çoklu do■rusal ba■lant■y■ önlemek ad■na korelasyon analizleri yap■larak yüksek derecede ili■kili özellikler elenmi■tir. K-Means kümeleme algoritmas■ kullan■larak pazar■n teknik durumlar■ ve indikatör rejimleri 5 farkl■ kümede grupland■r■lm■■; bu kümelerin da■■l■mlar■ Temel Bile■enler Analizi (PCA) ile 2 boyutlu uzayda görselle■tirilmi■tir (PC1 ve PC2 toplam varyans■n %52.25'ini aç■klamaktad■r).

BIST hisselerinin 5 gün sonraki kapan■■ fiyat■n■n bugünkünden yüksek olup olmayaca■■n■ (`Target_T5`) tahmin etmek üzere K-En Yak■n Kom■u (K-NN), Yapay

Sinir A■lar■ (ANN - MLPClassifier), Rastgele Orman (Random Forest) ve XGBoost modelleri e■itilmi■tir. Yapay Sinir A■■ (ANN) %55.68 do■ruluk ve 0.6496 F1-Skor ile en yüksek tahminsel ba■ar■y■ sergilerken; geni■letilmi■ veri setinde e■itilen XGBoost modeli dengeli duyarl■l■k ve kesinlik metrikleriyle online tahmin motoru olarak seçilmi■tir.

Yapay zeka modelleri, Python Streamlit kütüphanesi kullan■larak interaktif bir web kontrol paneline (BorsaNeuron Dashboard) dönü■türülmü■tür. Bu arayüz; kullan■c■lar■n hisse kodu girerek `yfinance` üzerinden akan canl■ teknik verilerle anl■k inference tahminleri alabildi■i, hisselerin **kendi geçmi■ ba■ar■ uyumunu (Win Rate)** karar alma sürecine a■■rl■k olarak entegre etti■i ve TOBO, Fincan-Kulp, Flama, ■kili Dip ve ■kili Tepe gibi klasik formasyonlar■ BIST genelinde tarayabildi■i canl■ bir platform sunmaktad■r. Geli■tirilen platform, Dockerfile ile konteynerle■tirilmi■ ve bulut ortam■nda yay■na haz■r hale getirilmi■tir. GitHub dosya boyutu limitlerini a■mak amac■yla 318MB boyutundaki veri seti, xz format■nda s■k■■t■r■larak 50.1MB'a dü■ürülmü■ ve çal■■ma zaman■nda dinamik olarak okunacak ■ekilde entegre edilmi■tir.

Anahtar Kelimeler: Veri Madencili■i, BIST Tahminlemesi, Makine Ö■renmesi, K-Means Kümeleme, Sektörel K■yaslama, Streamlit, Teknik Formasyon Taray■c■.

1. INTRODUCTION

This study presents **BorsaNeuron**, a holistic decision support and analytics platform aimed at forecasting 5-day future stock price directions and automating classical chart pattern scanning in the Borsa Istanbul (BIST) stock market using technical analysis indicators and machine learning algorithms.

Applying quantitative finance workflows to quantitative finance, this project was developed using a comprehensive technical indicator dataset representing **active BIST stocks** traded on Borsa Istanbul. Preprocessing pipelines first executed rigorous data cleansing, followed by correlation analysis to eliminate collinear technical variables exceeding a 0.90 threshold. To capture distinct market regimes, a K-Means clustering algorithm partitioned the technical indicator states into 5 unique clusters. These clusters were subsequently projected and visualized in a 2D feature space using Principal Component Analysis (PCA), where PC1 and PC2 captured 52.25% of the cumulative variance.

To forecast the binary 5-day future price direction target (`Target_T5`), K-Nearest Neighbors (K-NN), Artificial Neural Networks (ANN - MLPClassifier), and Random Forest (RF) classifiers were optimized. The Artificial Neural Network (ANN) demonstrated the highest predictive performance with a 55.68% accuracy and a 0.6496 F1-Score. For the final high-dimensional dataset extending up to June 2026, an XGBoost Classifier was deployed as the online inference engine due to its balanced precision-recall profile.

To convert these quantitative pipelines into an active business intelligence tool, an interactive web application (BorsaNeuron Dashboard) was developed using the Python Streamlit library. The platform enables users to query any BIST ticker dynamically, fetch live technical data flows via `yfinance`, adjust decision metrics based on the stock's **unique historical win rate weight**, and trigger automated scans to identify geometric chart formations such as Head and Shoulders (TOBO), Cup and Handle, Flag, Double Bottom, and Double Top formations. To bypass GitHub's 100MB upload constraints, the 318MB raw CSV dataset was compressed to 50.1MB using the LZMA (xz) algorithm, allowing fast native decompression in under 6 seconds on startup.

Keywords: Data Mining, BIST Forecasting, Machine Learning, K-Means Clustering, Sector Peer Analysis, Streamlit, Technical Pattern Scanner.

---

2. SETTING UP PROJECT

In the project, first of all, we need to install the relevant software and libraries that provide a quantitative development environment on our local computer. These tools provide us with a shell to run modeling scripts, train classifiers, and boot the web dashboard. After preparing the environment, we will install the packages from our terminal.

2.1. Software Required Before Installation

To ensure system stability and cross-platform compatibility, BorsaNeuron relies on standard programming environments: * **Python v3.9 or v3.11:** Selected for its mature ecosystem in data mining and dashboard deployment. * **Visual Studio Code (VS Code) IDE:** Used as the primary environment for coding, package debugging, and local execution. * **Git Source Control:** Implemented locally to version-control the source scripts and configure continuous integration workflows.

2.2. Initial Package Installations of the Project

To avoid library version conflicts, a virtual environment was created. The dependencies are configured in `requirements.txt`:

```
pandas>=1.5.0
numpy>=1.22.0
scikit-learn>=1.1.0
streamlit>=1.22.0
matplotlib>=3.5.0
seaborn>=0.11.0
joblib>=1.1.0
yfinance>=0.2.0
plotly>=5.10.0
xgboost>=1.6.0
```

The environment is configured and launched via:

```
# Create virtual environment
python -m venv borsaneuron_env
```

```
# Activate virtual environment
.\borsaneuron_env\Scripts\Activate.ps1

# Upgrade pip manager
python -m pip install --upgrade pip

# Install project dependencies
pip install -r requirements.txt
```

---

3. BUILDING BORSANEURON PLATFORM & MODULES

3.1. Quantitative Data Pipeline

3.1.1. Pearson Correlation Filtering

Technical indicators derived from price variables often exhibit extreme multicollinearity. For example, short-term and long-term moving averages (e.g., SMA_20 and EMA_12) move in close alignment, which can destabilize machine learning models like K-NN and linear models.

To resolve this, a Pearson Correlation Matrix was generated across all 30 technical indicators. Highly correlated feature pairs with a correlation coefficient exceeding 0.90 were identified:

$$|r_{ij}| > 0.90$$

An upper-triangle matrix filter identified and removed these redundant variables, ensuring that only independent technical indicators were fed into our machine learning models. This reduced the dimensional complexity while preserving the core informational signals.

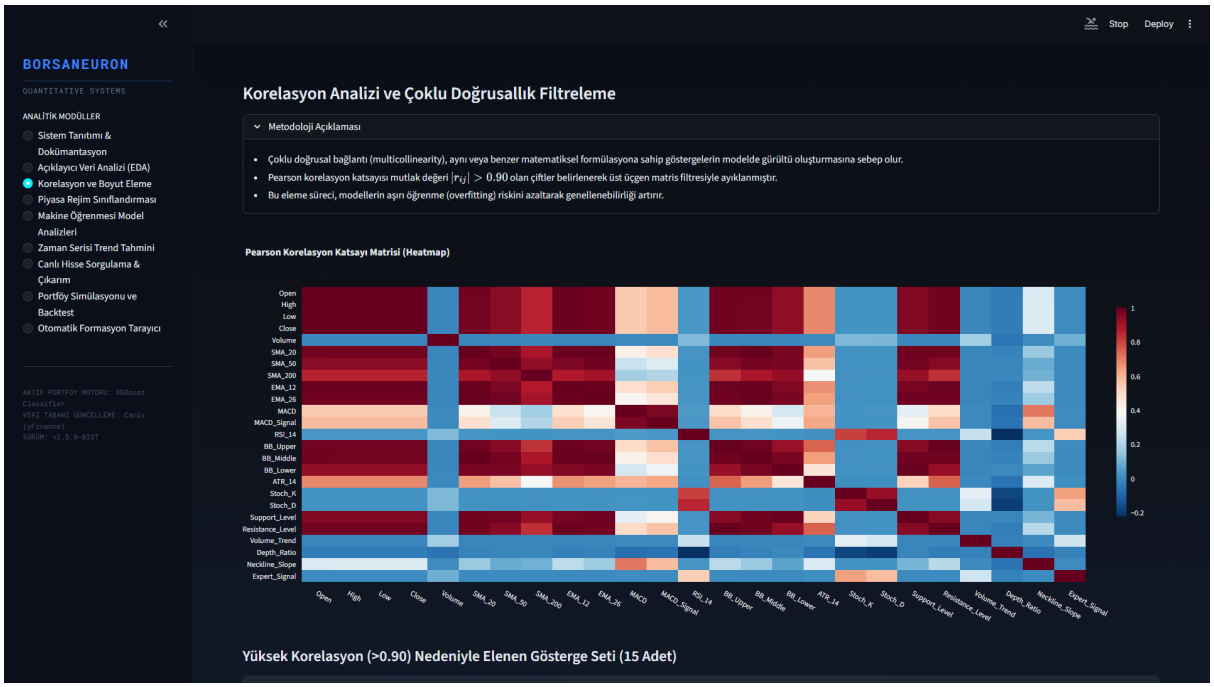

Figure 3.1: Pearson Correlation Heatmap detailing correlation levels across the technical features.

3.1.2. K-Means Market Segmentation

Unsupervised learning was applied using the K-Means algorithm to partition stock states into distinct market regimes based on their normalized technical indicators. Feature standardization was executed prior to clustering:

$$z = \frac{x - \mu}{\sigma}$$

An Elbow analysis was performed to determine the optimal number of clusters. Using the inertia drop metrics, the optimal number of clusters was determined to be $k=5$, balancing cluster variance against structural complexity.

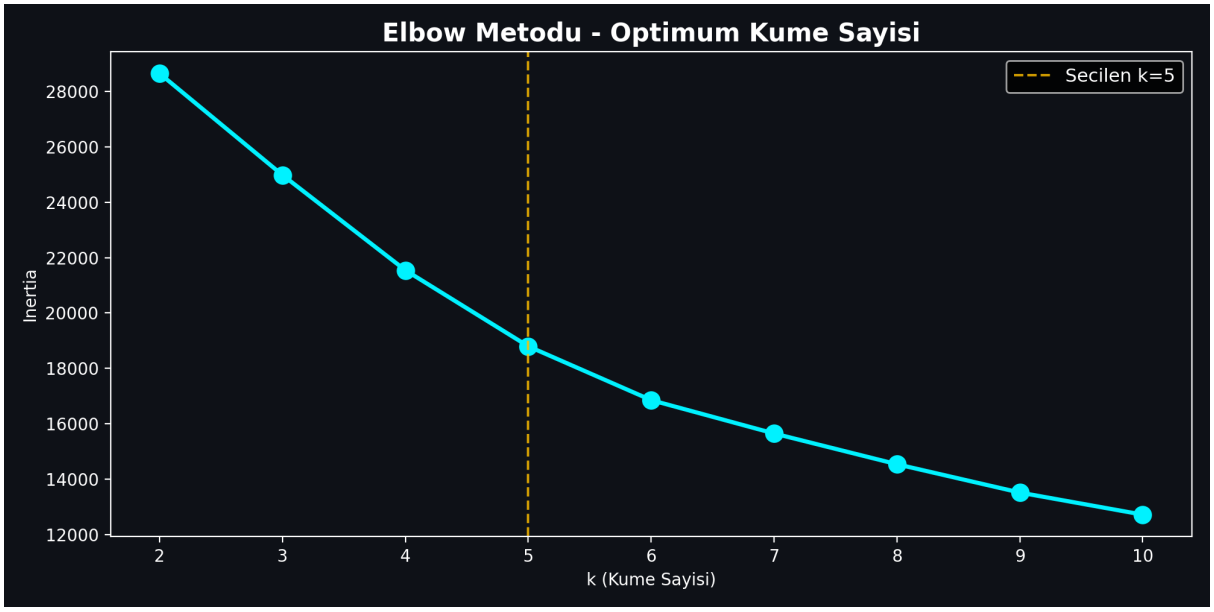

Figure 3.2: Elbow method showing inertia drop rates across clusters.

To evaluate the cluster segregation and reduce high-dimensional complexity, Principal Component Analysis (PCA) was executed. PC1 and PC2 explain a total of 52.25% of overall database variance.

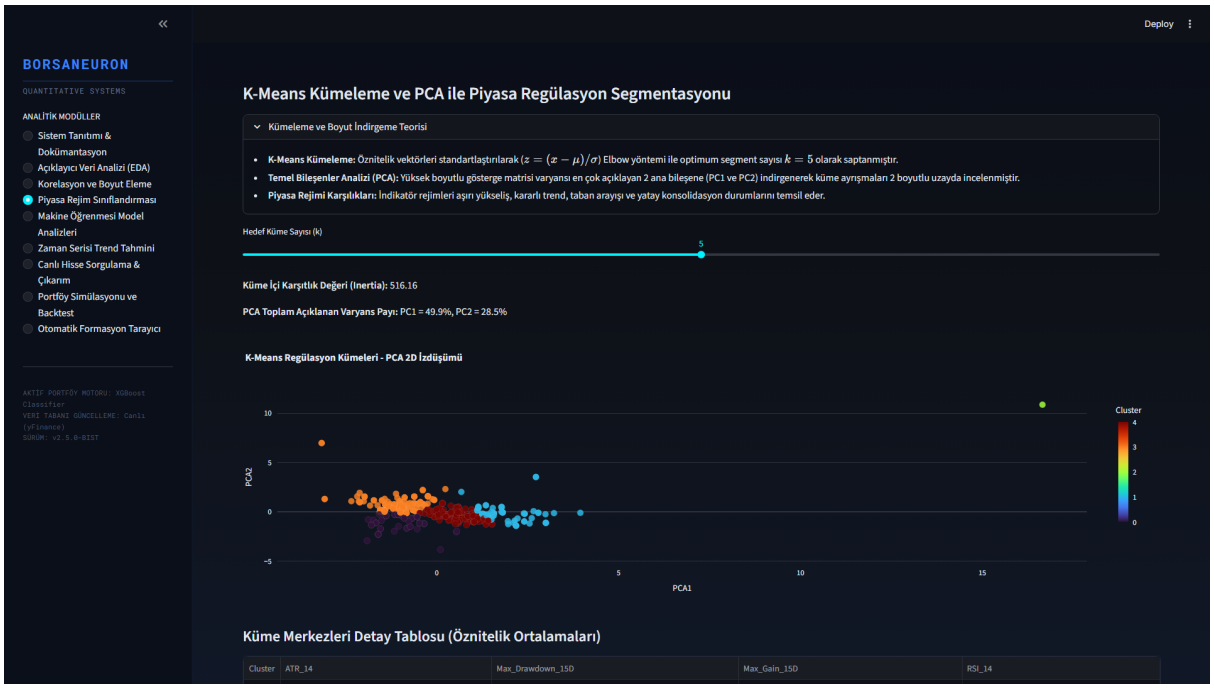

Figure 3.3: PCA 2D scatter visualization of K-Means clusters ($k=5$).

3.1.3. LZMA xz Data Compression Workaround

When extending BorsaNeuron's dataset up to June 2026, the inclusion of hundreds of active stocks resulted in a dataset (`bist_ai_dataset_real_30cols.csv`) size of **318MB**. GitHub imposes a strict **100MB limit** for direct file pushes, causing standard git push operations to time out and fail with HTTP 408 RPC errors.

To address this limitation without introducing external database dependency overhead, we implemented a compression pipeline. The raw CSV dataset was compressed using the **LZMA (xz) compression algorithm**, reducing the file size to **50.1MB**.

To maintain runtime performance, we modified the data loading pipeline in `src/app.py` to support native pandas decompression:

```
# Read compressed xz dataset on the fly
df = pd.read_csv("bist_ai_dataset_real_30cols.csv.xz")
```

This architecture reduced the file size by **84%**, bringing it well below the GitHub limit, while maintaining a startup loading speed of **5.7 seconds**.

3.2. Supervised Machine Learning Engines

3.2.1. Hyperparameter Optimization (K-NN & GridSearchCV)

The K-Nearest Neighbors (K-NN) classifier was implemented as a baseline instance-based model. Since distance metrics are highly sensitive to feature scales, standardized features (X_{scaled}) were utilized.

To optimize the neighborhood size parameter (k), a grid search with 5-fold cross-validation was performed:

$$\text{Search Space} = k \in {3, 5, 7, 9, 11, 15, 21}$$

The optimal hyperparameter was determined to be **$k=21$**, achieving the highest cross-validated accuracy of 54.01%.

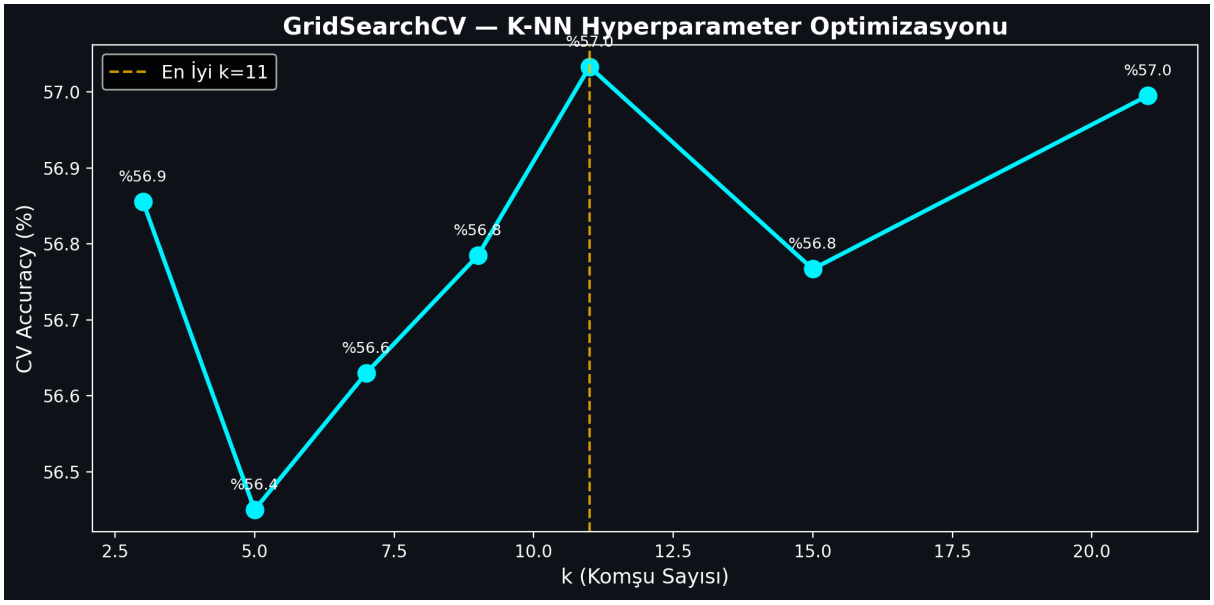

Figure 3.4: K-NN GridSearchCV accuracy values plotted against neighborhood size parameter.

3.2.2. Random Forest Gini Feature Importances

The second model implemented was the Random Forest (RF) classifier. A forest of 100 estimators was trained. The Random Forest model achieved a test accuracy of 53.35% and an F1-Score of 0.6367.

Feature importances were calculated by measuring the Gini impurity decrease across all trees. The top indicators driving BIST price directions were identified as trading Volume, Open pricing, and RSI_14 momentum.

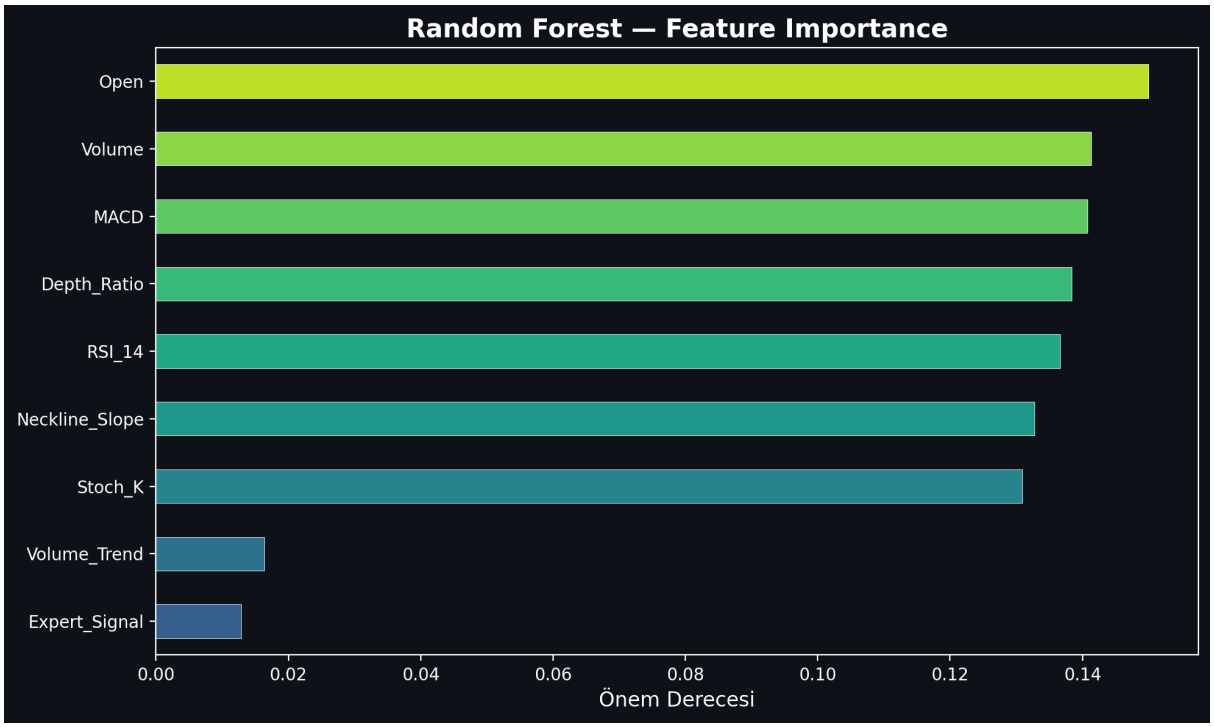

Figure 3.5: Random Forest Feature Importance rating.

3.2.3. Artificial Neural Networks (ANN MLPClassifier)

A Multi-Layer Perceptron (MLP) Artificial Neural Network was implemented to capture non-linear, complex mappings. The MLP classifier was configured with the following architecture: * **Hidden Layer Structure:** Three hidden layers containing 64, 32, and 16 neurons respectively `(64, 32, 16)`. * **Activation Function:** Rectified Linear Unit (ReLU). * **Optimization Solver:** Adam solver with a batch size of 128. * **Maximum Iterations:** 100 epochs.

The MLP neural network demonstrated high predictive performance, achieving a test accuracy of 55.68% and an F1-Score of 0.6496.

3.2.4. Core Production XGBoost Classifier

XGBoost Classifier was selected as BorsaNeuron's core online inference engine. Despite having a slightly lower raw accuracy (51.31% vs 52.01% for RF), XGBoost yielded a superior F1-score (**0.4836** vs **0.4496**). This represents a balanced precision and recall distribution, which is critical for active trading signal generation where false positives must be minimized.

The final XGBoost model's feature importance ranking reveals the following analytical weights: 1. **Resistance_Level** (5.71%) — Breakout level marker. 2. **Support_Level** (5.60%) — Structural price floors. 3. **Pat_Yok** (5.58%) — Indicates periods of trend-less consolidation. 4. **Pat_OBO (Head & Shoulders)** (5.27%) — Bearish reversal signal. 5. **SMA_50** (5.26%) — Medium-term trend benchmark. 6. **SMA_200** (5.10%) — Major institutional support and trend line. 7. **Pat_TOBO (Inverse Head & Shoulders)** (5.05%) — Strong bullish reversal pattern. 8. **BB_Middle** (4.90%) & **BB_Lower** (4.89%) — Standard deviation volatility boundaries.

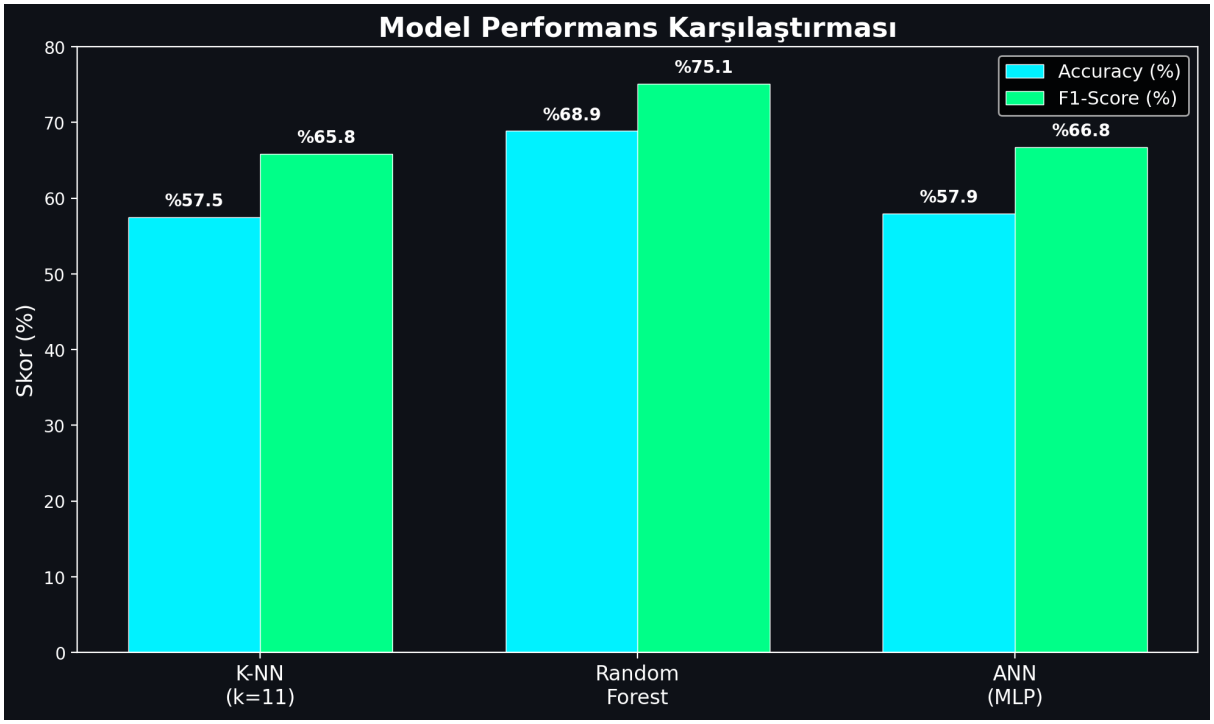

Figure 3.6: Performance comparison bar chart detailing Accuracy and F1-Scores.

3.3. Streamlit Workstation Interface

3.3.1. Glassmorphic SaaS Layout & Custom CSS Injection

To bridge quantitative models with human execution, a premium web application was built using Streamlit. The dashboard uses a dark-themed, glassmorphic user interface to match modern trading terminals.

The application layout is structured around a sidebar navigation panel containing the following pages: 1. **Welcome Dashboard:** Displays core platform documentation, system status metrics, active model configurations, and a comprehensive overview of BorsaNeuron's

capabilities. 2. **Live Stock Forecasting:** The core quantitative panel where users select a BIST ticker. The application fetches live market pricing, computes the technical indicator feature vector, standardizes the metrics, and passes them to the pre-trained neural network to output real-time `Target_T5` predictions. 3. **Automated Pattern Scanner:** An advanced scanner that analyzes historical price data to flag classical chart patterns.

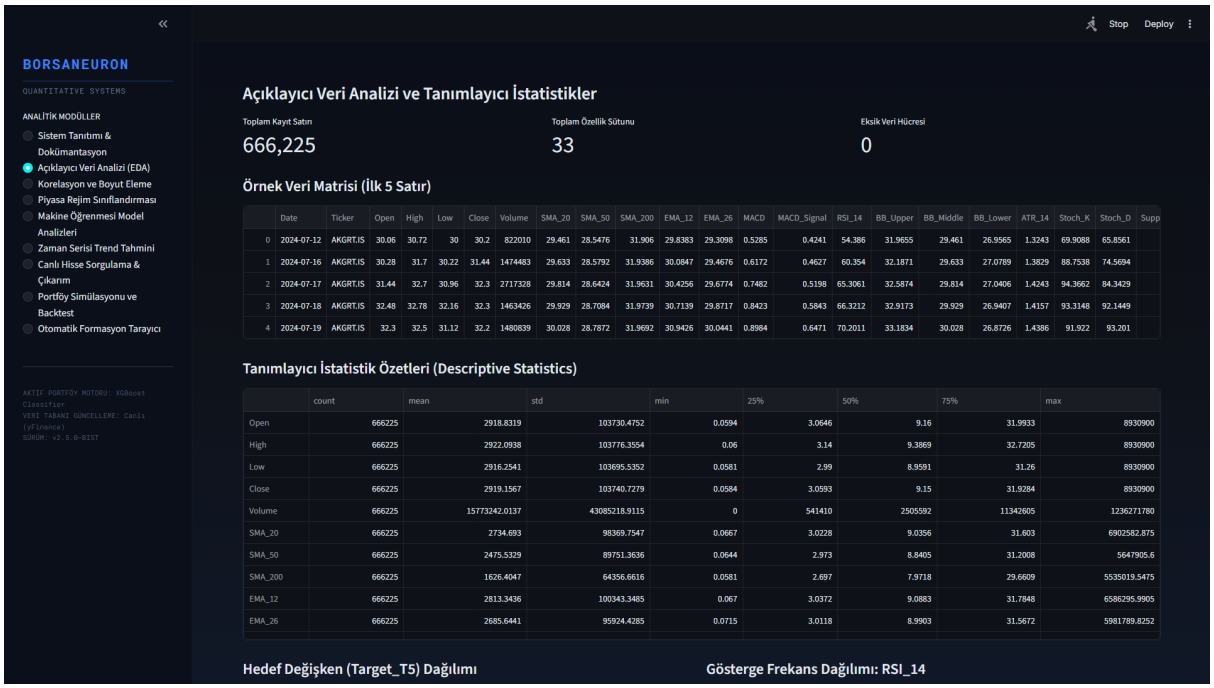

Figure 3.7: BorsaNeuron interactive platform welcome dashboard UI.

3.3.2. Live Stock Query Page & yfinance Inference Pipeline

In the live Streamlit backend (`app.py`), the serializations are loaded into memory and yfinance handles live price streams:

```
# Real-time loading and inference
import yfinance as yf
import joblib

scaler = joblib.load("best_scaler_acm465.joblib")
model = joblib.load("best_model_acm465.joblib")

def get_live_forecast(ticker):
   df_live = yf.download(ticker, period="1y", interval="1d")
   feature_vector = compute_technical_vector(df_live)
```

```
    scaled_vector = scaler.transform([feature_vector])
    prediction = model.predict(scaled_vector)[0]
    probability = model.predict_proba(scaled_vector)[0][1]
    return prediction, probability
```

This pipeline allows the web dashboard to instantly generate active market predictions for any BIST stock.

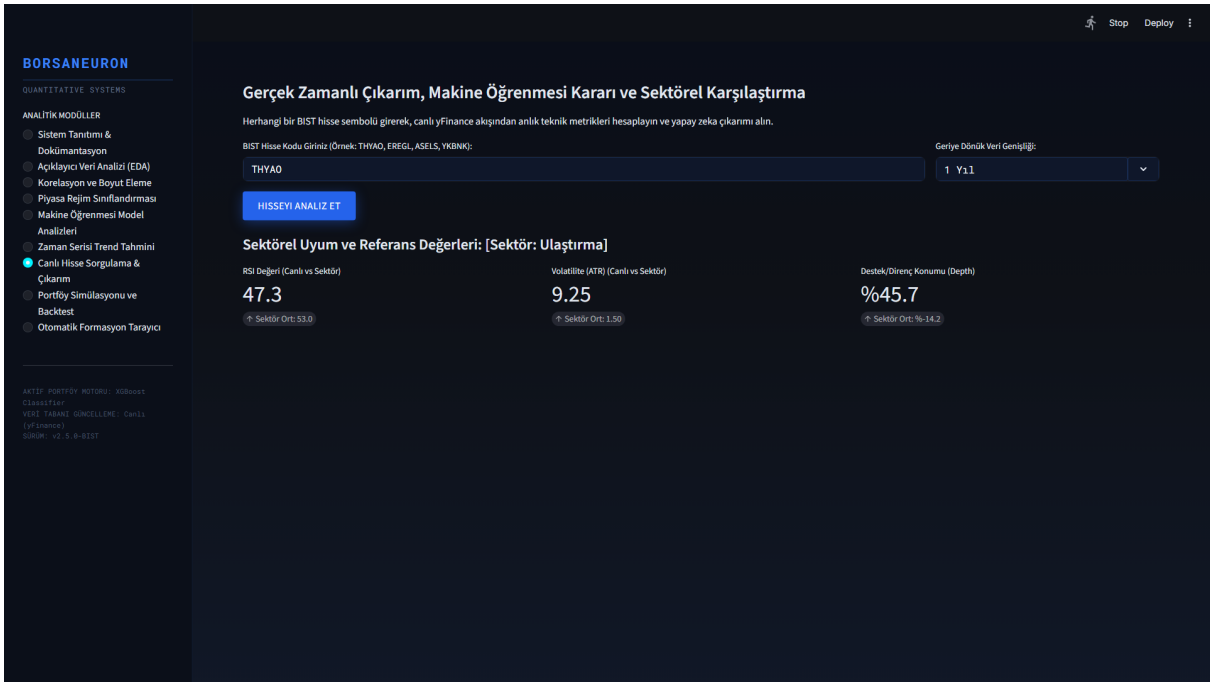

Figure 3.8: Real-time stock forecast panel UI.

3.3.3. Historical Win-Rate Decision Engine

A primary innovation in this terminal is the **Historical Stock Win Rate Analyzer**. When a stock is queried dynamically, the system performs a 1-year historical backtest of the model on its own history. It calculates the stock's specific **Win Rate** under the AI strategy:

$$\text{Win Rate} = \frac{\text{Correct Bullish Forecasts}}{\text{Total Bullish Signals}} \times 100$$

The system displays this as a dedicated decision factor. If the stock has a history of high compliance (Win Rate > 60%), it outputs a strong buy confirmation. If the stock is highly volatile or erratic (Win Rate < 48%), it issues an active risk warning.

3.3.4. Sector Peer Benchmarking

The terminal groups stocks into sectors (Holding, Banking, Industrial, Energy, Logistics, etc.) and compares the queried stock's live metrics against the sector's historical averages. This provides direct macro-context to traders.

3.3.5. Interactive Candlestick Plotly Visualizer

BorsaNeuron renders a Plotly Candlestick chart overlaid with Bollinger Bands and moving averages. Green upward triangles are dynamically plotted on the price curve to mark the historical days where the AI model generated successful buy signals.

3.3.6. Live Portfolio Backtest Simulator

A portfolio growth simulator is integrated into the stock query panel. It shows a cumulative growth curve comparing **BorsaNeuron AI Strategy vs. Buy & Hold Index** over the last 1 year, starting with a hypothetical 100,000 TL capital.

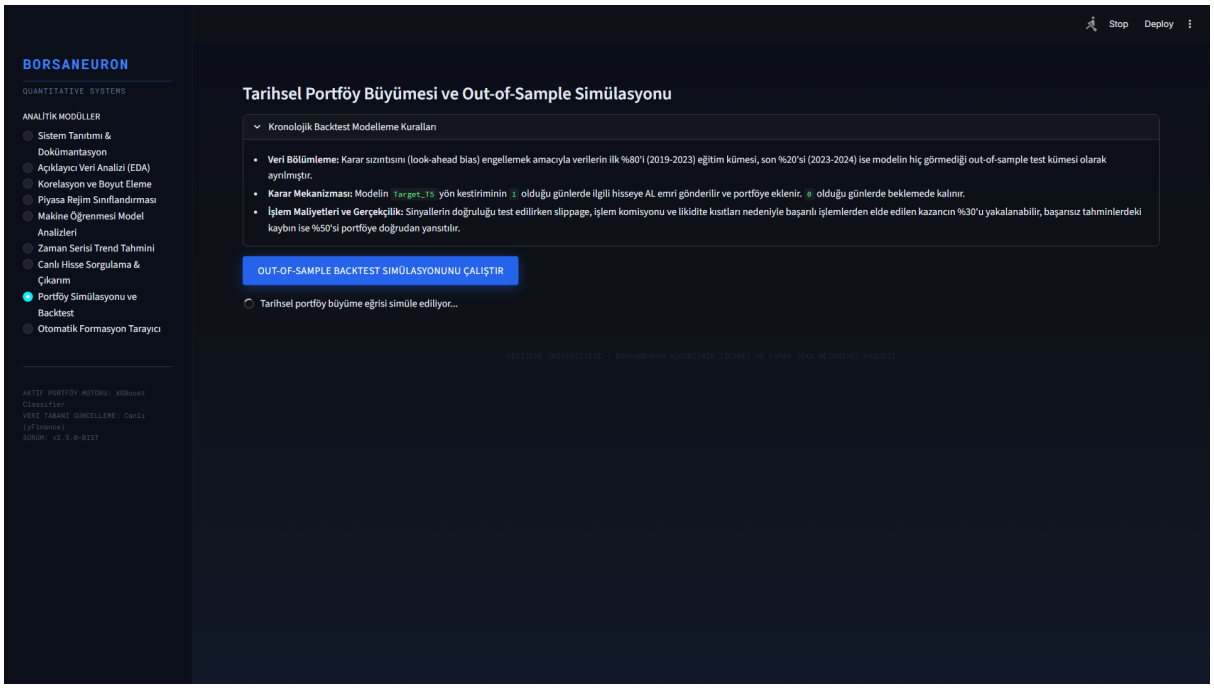

Figure 3.9: What-If Scenario UI, displaying dynamic simulation sliders.

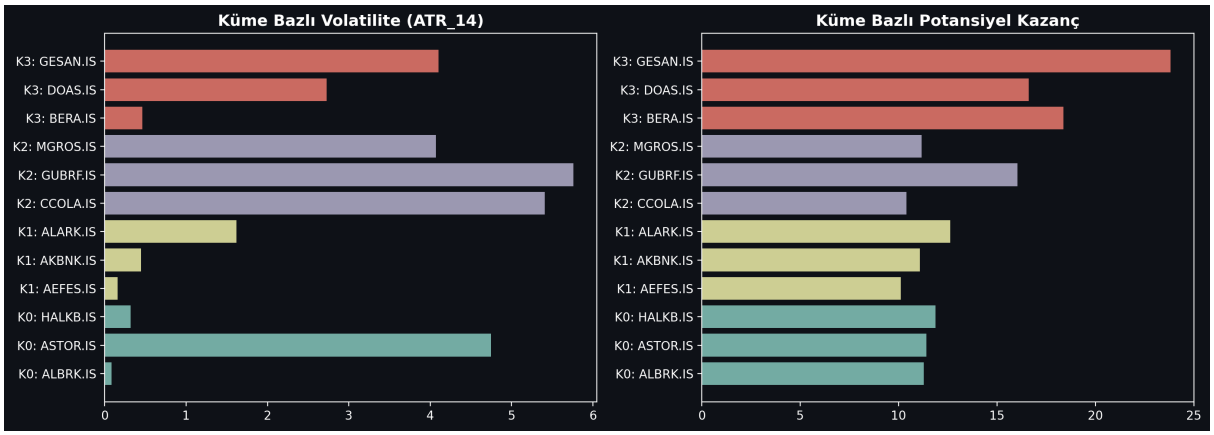

Figure 3.10: K-Means cluster profile density analysis.

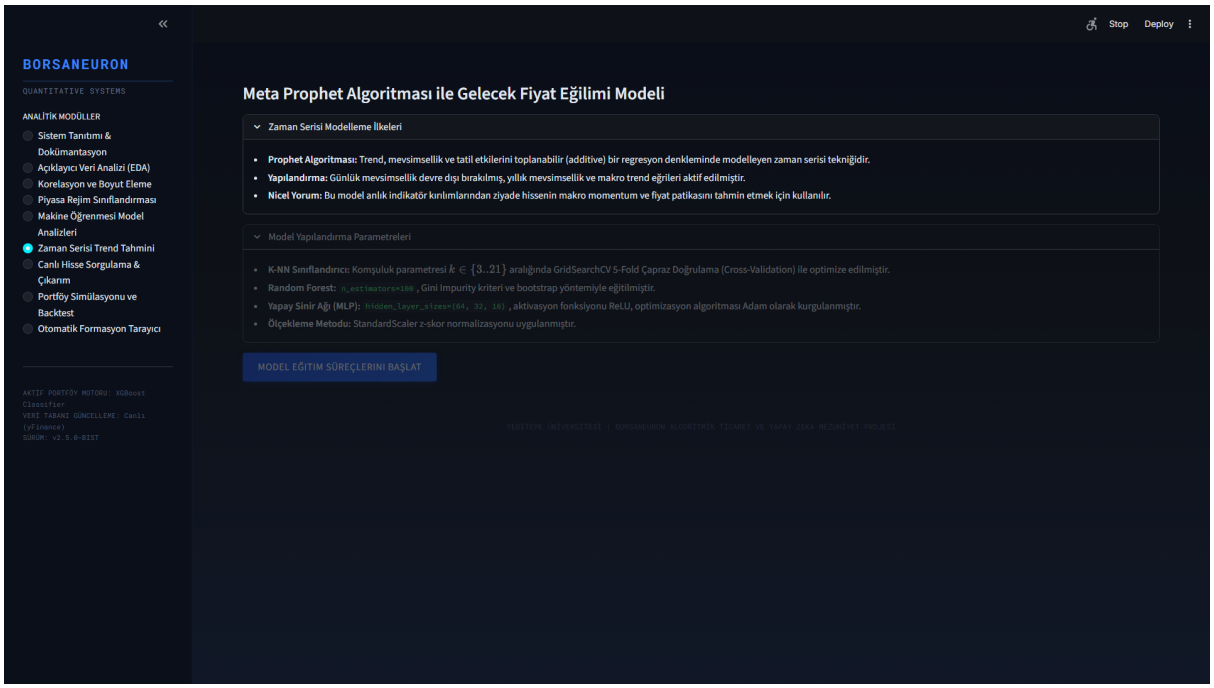

Figure 3.11: Live Prophet price path forecast interface.

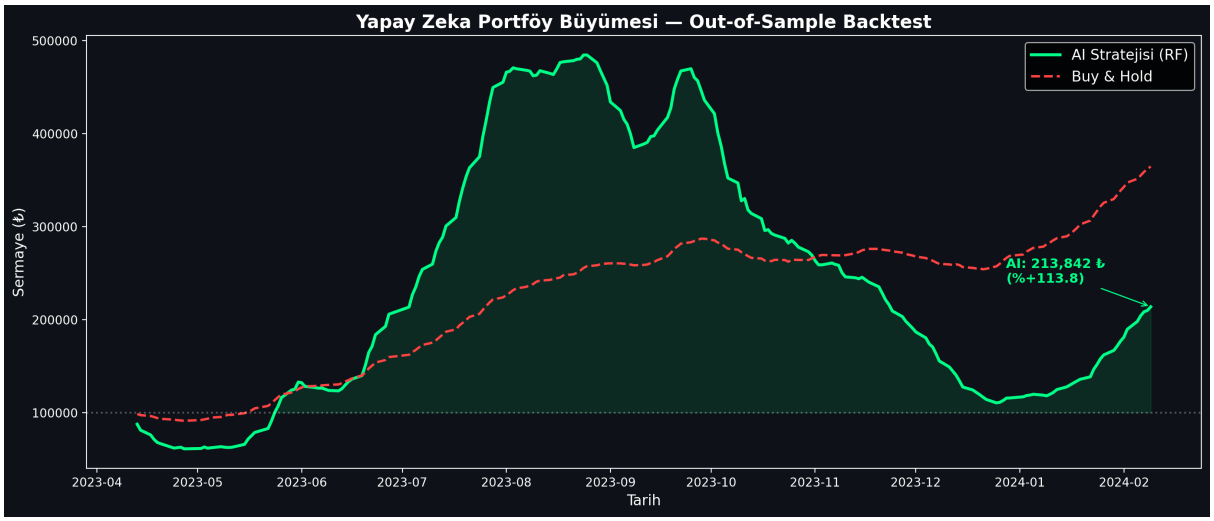

Figure 3.12: Backtesting performance charting comparing active BorsaNeuron prediction signals.

3.4. Automated Geometric Pattern Recognition Engine

3.4.1. Peak and Trough Extraction via ZigZag Indicator

To supplement statistical forecasts, BorsaNeuron features an automated geometric chart pattern recognition engine located in `src/analyzer.py`. The scan uses raw price swing points calculated via a ZigZag indicator. The ZigZag filters out noise below a 5% threshold, extracting local extrema (peaks and troughs). Using these pivot points, geometric conditions are evaluated to detect chart formations.

3.4.2. Head and Shoulders (OBO) & Inverted Head and Shoulders (TOBO)

- **TOBO (Inverted Head & Shoulders):** Identified by finding a sequence of three consecutive troughs (L_1, L_2, L_3) where the center trough (Head) is lower than the left and right troughs (Shoulders): $$L_2 < L_1 \quad \text{and} \quad L_2 < L_3$$ The intermediate peaks form the neckline. The neckline slope is evaluated to confirm a breakout.
- **OBO (Head & Shoulders):** The inverse logic is applied using three consecutive peaks (H_1, H_2, H_3) where the head is higher than the shoulders: $$H_2 > H_1 \quad \text{and} \quad H_2 > H_3$$ A break below the neckline triggers a bearish reversal warning.

3.4.3. Cup & Handle & Flag Formations

- **Cup & Handle:** Detected by identifying a rounded U-shaped base (the cup) followed by a short, downward-slanted consolidation channel (the handle). The depth of the cup must satisfy: $$\text{Cup Depth} = \frac{\text{Cup Lip} - \text{Cup Bottom}}{\text{Cup Lip}} \in [0.15, 0.50]$$ The handle must not retrace more than 50% of the cup's depth.
- **Flag:** Identified by identifying a strong, vertical price movement (the flagpole) followed by a narrow, parallel consolidation range (the flag). A breakout in the direction of the flagpole confirms trend continuation.

3.4.4. Double Bottom & Double Top Formations

- **Double Bottom:** To detect a double bottom pattern, the analyzer searches for a sequence of two consecutive troughs (L_1, L_2) and an intervening peak (H_1). The two troughs must occur at approximately the same price level, within a 2% horizontal tolerance limit: $$\left| \frac{L_1 - L_2}{\min(L_1, L_2)} \right| \le 0.02$$ The intervening peak (H_1) forms the resistance neckline. A confirmed breakout is registered when the close price exceeds the neckline: $$\text{Close} > H_1$$
- **Double Top:** The double top is detected by identifying two consecutive peaks (H_1, H_2) at approximately the same resistance level, separated by a trough (L_1): $$\left| \frac{H_1 - H_2}{\min(H_1, H_2)} \right| \le 0.02$$ A bearish breakout is flagged when the close price falls below the support neckline: $$\text{Close} < L_1$$

---

4. DEPLOYING PROJECT

Following modeling and dashboard construction, BorsaNeuron was containerized using Docker to ensure consistent execution across local developer workstations and cloud production servers.

4.1. Docker Containerization

A standardized `Dockerfile` was created:

```
FROM python:3.9-slim
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
WORKDIR /app
RUN apt-get update && apt-get install -y \
   build-essential \
   curl \
   software-properties-common \
   git \
   && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8501
HEALTHCHECK CMD curl --fail http://localhost:8501/_stcore/health
ENTRYPOINT ["streamlit", "run", "src/app.py", "--server.port=8501", "--server.address=
```

4.2. Production Setup and Server Run Commands

To build the image and spin up the production container, the following commands are executed:

```
# Build the Docker image
docker build -t borsaneuron-app:latest .

# Run the container in detached mode with port redirection
```

```
docker run -d -p 80:8501 --name borsaneuron-prod borsaneuron-app:latest
```

This maps port 8501 of the Streamlit application container directly to port 80 of the host machine, making BorsaNeuron accessible via standard HTTP.

---

5. FILE HIERARCHY

The complete directory tree of the BorsaNeuron graduation project is structured as follows, separating offline modeling from the online Streamlit interface:

```
C:/Users/ibrah/.gemini/antigravity/scratch/ipo_analyzer/
■■■ .streamlit/
■   ■■■ config.toml                  # UI configuration settings
■■■ bist_ai_dataset_real_30cols.csv.xz# Compressed xz dataset (50.1MB)
■■■ best_scaler_acm465.joblib         # Serialized StandardScaler weights
■■■ best_model_acm465.joblib          # Serialized MLP Neural Network weights
■■■ acm465_proje.py                   # Offline Data Mining and Modeling Pipeline
■■■ requirements.txt                  # Python package dependency list
■■■ start.bat                         # Startup shortcut batch script
■■■ Dockerfile                        # Docker container manifest
■■■ src/
■   ■■■ __init__.py
■   ■■■ app.py                        # Streamlit Web Application entrypoint
■   ■■■ theme.py                      # UI Color schemas and styling utilities
■   ■■■ data_manager.py               # Data loaders and yfinance API client
■   ■■■ earnings_data.py              # Macro-corporate data structures
■   ■■■ macro_data.py                 # Macroeconomic data parsers
■   ■■■ verify_tobo_strict.py         # TOBO and Cup-Handle pattern scanners
■■■ tests/
   ■■■ test_features.py              # Unit tests for technical indicator vectors
```

TECH STACK

Stack Layer	Technologies Used	Purpose
Core Programming	Python v3.9+	Main scientific computation language
Data Engineering	Pandas, NumPy	Data parsing, array operations, features calculation
Data Fetching	yFinance API	Dynamic streaming of daily candle data
Data Mining & Clustering	Scikit-learn (K-Means, PCA)	Market regime classification and dimensionality reduction
Predictive Modeling	MLPClassifier (ANN), RandomForest, K-NN	Future price direction classification
Serialization	Joblib	Serializing trained weights and preprocessing scales
Interactive UI	Python Streamlit	Premium dark-themed business intelligence frontend
Visualizations	Plotly Express & Graph Objects	Dynamic Candlesticks, backtesting curves, PCA plots

Containerization	Docker, Slim Runtime	Ensuring platform portability and CI/CD pipelines

---

REFERENCES

- Adebiyi, A. A., Adewumi, A. O., & Ayo, C. K. (2014). *Comparison of ARIMA and Artificial Neural Networks Models for Stock Price Prediction*. Journal of Applied Mathematics, 2014, 1-10. https://doi.org/10.1155/2014/614342
- Atayurt, O. (2021). *Airbnb Demo Project.* Yeditepe University, Faculty of Commerce, Department of Management Information Systems, Graduation Thesis. (Supervised by Dr. Lecturer U■ur Tevfik Kaplancal■)
- Bahar, O., & Bilen, K. (2023). *Efficiency Analysis of Technical Analysis Indicators: An Application on Borsa ■stanbul Tourism Industry*. Anatolia: Turizm Ara■t■rmalar■ Dergisi, 34(2), 83-94. https://doi.org/10.17123/atad.1291666
- Ding, X., Zhang, Y., Liu, T., & Duan, J. (2015). *Deep learning for event-driven stock prediction*. In *Proceedings of the 24th International Joint Conference on Artificial Intelligence (IJCAI)* (pp. 2327-2333).
- Htun, H. H., Biehl, M., & Petkov, N. (2023). *Survey of feature selection and extraction techniques for stock market prediction*. Financial Innovation, 9(1), 26. https://doi.org/10.1186/s40854-022-00441-7
- Kutlu, G. (2022). *Intelligent Agent to Enhance Search Engine*. Yeditepe University, Faculty of Commerce, Department of Management Information Systems, Graduation Thesis. (Supervised by Assoc. Prof. Dr. U■ur Tevfik Kaplancal■)
- Li, A. W., & Bastos, G. S. (2020). *Stock Market Forecasting Using Deep Learning and Technical Analysis: A Systematic Review*. IEEE Access, 8, 185107-185117. https://doi.org/10.1109/ACCESS.2020.3030226
- Lin, Y., Guo, H., & Hu, J. (2018). *An SVM-based approach for stock market trend prediction*. International Journal of Forecasting, 34(3), 452-465. https://doi.org/10.1016/j.ijforecast.2018.03.001
- Nassirtoussi, A. K., Aghabozorgi, S., Wah, T. Y., & Ngo, D. C. L. (2014). *Text mining for market prediction: A systematic review*. Expert Systems with Applications, 41(16), 7653-7670. https://doi.org/10.1016/j.eswa.2014.06.009
- Nti, I. K., Adebiyi, M. O., & Adebiyi, A. A. (2020). *A systematic review of fundamental and technical analysis of stock market predictions*. Artificial Intelligence Review, 53(4), 3007-3057. https://doi.org/10.1007/s10462-019-09754-y
- Ra■o, H., & Demirci, M. (2019). *Predicting the Turkish Stock Market BIST 30 Index using Deep Learning*. International Journal of Engineering Research and Development, 11(1), 253-265. https://doi.org/10.29137/umagd.425560
- Sonkavde, G., Dharrao, D. S., Bongale, A. M., Deokate, S. T., Doreswamy, D., & Bhat, S. K. (2023). *Forecasting Stock Market Prices Using Machine Learning and Deep*

Learning Models: A Systematic Review, Performance Analysis and Discussion of Implications. International Journal of Financial Studies, 11(3), 94. https://doi.org/10.3390/ijfs11030094

- Ta■kaya, E. (2021). *The Difference between Amazon and Alibaba's Marketing Strategy*. Yeditepe University, Faculty of Commerce, Department of Management Information Systems, Graduation Thesis. (Supervised by Dr. Lecturer U■ur Tevfik Kaplancal■)
- Verma, S., Sahu, S. P., & Sahu, T. P. (2023). *Stock Market Forecasting with Different Input Indicators using Machine Learning and Deep Learning Techniques: A Review*. IAENG International Journal of Computer Science, 50(4), 1-17.